

Python の基礎

目次

Python の基礎	1
この章の目標と進め方	1
準備：ノートブックを開く	2
計算と変数	2
計算を表示する	2
値に名前を付ける	3
演習 1：予算を計算する	3
数値・文字列・真偽値	3
演習 2：文字を数値にする	4
リスト：複数の値をまとめる	4
演習 3：一つの値で平均はどう変わるか	4
条件分岐：条件で処理を変える	4
演習 4：境界を確かめる	5
繰り返し：同じ処理を各データに行う	5
条件に合う人数を数える	5
演習 5：条件に合う値を集める	5
辞書：項目名と値を対応させる	5
関数：処理に名前を付ける	6
演習 6：割合を求める関数	6
エラーを読んで直す	6
総合課題：学習時間の集計メモ	7
解答例と振り返り	7
演習 1・2	8
演習 3・4	8
演習 5・6	8
総合課題の解答例	8
公式資料	9

Python の基礎

この章の目標と進め方

Python は、計算やデータ処理の手順を文章のようなコードで書く言語です。この章では、架空の 5 人の学習時間を使い、集計して説明するところまで進みます。プログラミングの経験は必要ありません。

- ・ 値に名前を付け、数値と文字列を区別する。
- ・ リストと辞書で複数のデータを扱う。
- ・ 条件分岐・繰り返し・関数を組み合わせる。
- ・ エラーを読み、上から再実行できるノートブックを作る。

授業全 15 回・サブゼミ全 10 回のうち、**授業 2 回（各 90 分を仮定） + サブゼミ 1 回（60 分）** を使う案です。授業時間に合わせて途中の演習を宿題にできます。

回	内容	配分の目安
授業 A	実行環境・計算・変数・リスト	準備 15 分、説明と実行 45 分、演習 20 分、振り返り 10 分

回	内容	配分の目安
授業 B	条件分岐・繰り返し・辞書・関数	復習 10 分、説明と実行 55 分、演習 20 分、振り返り 5 分
サブゼミ	エラー修正・総合課題・相互確認	修正 15 分、課題 30 分、確認 15 分

各例は「結果を予想する → 実行する → 数値を一つ変える → 違いを説明する」の順に進めてください。解答例は章末にまとめています。

準備：ノートブックを開く

ノートブックは、説明文・コード・実行結果を一緒に保存するファイルです。拡張子は `.ipynb` です。

この章では準備の負担を小さくするため、ブラウザで動く **Google Colab** を使います。Google アカウントとネット接続が必要です。VS Code で Python と Jupyter が使える人は、同じファイルを VS Code でも開けます。

1. 演習ノートブックをダウンロードします。
2. Google Colab を開き、サインインします。
3. ノートブックを開く画面の「アップロード」からファイルを選びます。
4. 必要に応じて自分の Drive にコピーを保存します。保存先とファイル名を確認します。
5. 最初のコードセルを選び、左の実行ボタン、または Shift+Enter で実行します。

「セル」はノートブックの一つの入力枠です。コードセルに Python、テキストセルに Markdown を書きま。教材 Web ページ上のコードは、その場では実行されません。

```
print("こんにちは、Python")
```

結果： `こんにちは、Python` と表示されます。`print()` は括弧内の値を表示する関数です。引用符は文字列の境界を表します。

i 実行順序が大切

上のセルで作った変数を下のセルで使います。画面の並びではなく、実際に実行した順番で状態が変わります。作業の最後はランタイム（実行環境）を再起動し、「すべてのセルを実行」で上から動くか確認します。Colab の接続が切れると変数が失われることがあるので、ノートブックの保存を確認しましょう。

VS Code で実行する場合は、Python 環境と Microsoft の Python・Jupyter 拡張を準備し、右上のカーネル選択で使用する Python を選びます。カーネルはコードを実行するプログラムです。未設定なら今回は Colab で進めて構いません。VS Code 公式手順も参照できます。

計算と変数

計算を表示する

```
print(1200 + 800)
print(1200 * 3)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(2 ** 3)
```

結果（上から）： 2000、3600、3.5、3、1、8。

* は掛け算、/ は割り算、// は商を小さい整数側へ丸める除算、% は余り、** は累乗です。掛け算・割り算は足し算より先に計算されます。順序を変えたいときは括弧を使います。

値に名前を付ける

```
price = 1200
quantity = 3
total = price * quantity
print(total)
```

結果：3600。price などを変数と呼びます。= は「右側を計算し、左側の名前に結び付ける」という代入です。数学の等式とは意味が異なります。

```
quantity = 4
print(total)
total = price * quantity
print(total)
```

結果：3600、4800。変数を変えても、過去に計算した値は自動更新されません。計算し直す必要があります。変数名には study_hours のように意味の分かる英小文字とアンダースコアを使います。大文字・小文字は区別されます。print、sum、list のような組み込み機能の名前は変数名に使わないでください。

演習 1：予算を計算する

1冊 850 円の本を 3 冊買い、送料が 200 円かかります。価格・冊数・送料をそれぞれ変数にして合計を求めてください。その後、冊数を 4 に変更して再計算します。結果が変わる理由を 1 文で説明してください。

数値・文字列・真偽値

型	例	用途
int	3	整数
float	2.5	小数を含む数値
str	"ゼミ"、"3"	文字列
bool	True、False	条件が成り立つか

```
hours_text = "3"
print(hours_text + "2")
hours = int(hours_text)
print(hours + 2)
print(type(hours_text).__name__)
```

結果：32、5、str。文字列の + は文字をつなぎます。int() は整数への変換、type() は型の確認です。小数の文字列 "2.5" を数値にしたいときは float() を使います。

```
name = "学生 A"
hours = 2.5
print(f"{name}の学習時間は{hours}時間です")
```

結果： 学生 A の学習時間は 2.5 時間です。先頭に f を付けた文字列では、{} に変数や計算式を入れられます。

から行末はコメントです。処理の目的や単位をメモできます。

```
weekly_hours = 6 # 単位は時間／週
print(weekly_hours >= 5)
```

結果： True。>= は「以上」の比較です。

演習 2：文字を数値にする

minutes_text = "90" を整数に変換し、60 で割って時間に直してください。f 文字列を使って 学習時間は 1.5 時間です と表示します。

リスト：複数の値をまとめる

5 人の 1 週間の学習時間を、リストにまとめます。角括弧 [] で囲み、値をコンマで区切ります。この章のデータはすべて架空です。

```
study_hours = [2, 5, 0, 3, 10]
print(study_hours[0])
print(study_hours[-1])
print(study_hours[1:3])
print(len(study_hours))
```

結果： 2、10、[5, 0]、5。位置を表す番号（インデックス）は 0 から始まります。-1 は最後、[1:3] は 1 番目から 3 番目の直前までです。len() は要素数を返します。

```
total_hours = sum(study_hours)
mean_hours = total_hours / len(study_hours)
print(total_hours)
print(mean_hours)
print(min(study_hours), max(study_hours))
```

結果： 20、4.0、0 10。sum() は合計、min() と max() は最小値・最大値です。平均は空のリストでは計算できません。

! 0 と欠測を区別する

ここでの 0 は「学習時間が 0 時間という回答」です。「未回答」を 0 に置き換えると平均の意味が変わります。実際のデータ分析では、欠測をどう扱ったかを必ず記録します。

演習 3：一つの値で平均はどう変わるか

study_hours を [2, 5, 0, 3, 20] に変えて平均を計算してください。最初の平均と比べ、平均だけで「典型的な学生」を説明する際の注意を 1 文で書きます。実験後は元の [2, 5, 0, 3, 10] に戻します。

条件分岐：条件で処理を変える

```
hours = 5
if hours >= 5:
    print("5 時間以上")
else:
    print("5 時間未満")
```

結果： 5 時間以上。if の後ろに条件を書き、行末に : を付けます。条件が成り立つと、その下の字下げした処理が実行されます。else はそれ以外です。字下げは半角スペース 4 個にそろえます。

比較	意味
a == b	等しい
a != b	等しくない
a > b、a >= b	より大きい、以上

比較	意味
<code>a < b</code> , <code>a <= b</code>	より小さい、以下

代入の `=` と比較の `==` を区別してください。条件を組み合わせるときは `and` (両方)、`or` (少なくとも一方) を使います。

```
hours = 0
if hours == 0:
    print("0 時間")
elif hours < 5:
    print("0 時間より多く 5 時間未満")
else:
    print("5 時間以上")
```

結果：0 時間。elif は追加の条件です。上から調べ、最初に成り立った枝だけを実行します。この例は 0 以上の時間を前提にしています。

演習 4：境界を確かめる

最初の if の例を、hours が 4、5、6 の場合で試してください。条件を `hours > 5` に変えると、どの場合だけ結果が変わりますか。

繰り返し：同じ処理を各データに行う

```
study_hours = [2, 5, 0, 3, 10]
for hours in study_hours:
    print(hours * 60)
```

結果（上から）：120、300、0、180、600。for はリストから値を順に取り出して hours に代入し、字下げした処理を繰り返します。

条件に合う人数を数える

```
count = 0
for hours in study_hours:
    if hours >= 5:
        count = count + 1
print(count)
```

結果：2。count = count + 1 は、現在の値に 1 を足して代入し直します。count の初期化はループの前、最後の表示はループの外です。実行前に、2、5、0、3、10 を読んだ直後の count を紙に書いてみてください。

range(3) は 0、1、2 を順に生成します。決まった回数の繰り返しで使えます。

```
for i in range(3):
    print(i)
```

演習 5：条件に合う値を集める

空のリスト selected = [] を用意し、5 時間以上の値を selected.append(hours) で追加してください。append() は末尾に要素を追加する操作です。元データを変更せず、結果が [5, 10] になることを確認します。

辞書：項目名と値を対応させる

辞書は、項目名（キー）で値を取り出す仕組みです。リストの「何番目」と区別しましょう。

```
student = {"id": "A", "hours": 2}
print(student["id"])
print(student["hours"])
```

結果： A、2。辞書をリストに入れると、小さな表のように複数人の情報を扱えます。

```
students = [
    {"id": "A", "hours": 2},
    {"id": "B", "hours": 5},
    {"id": "C", "hours": 0},
]
for student in students:
    print(f"{student['id']}: {student['hours']}時間")
```

結果： A: 2 時間、B: 5 時間、C: 0 時間。外側の二重引用符と内側の一重引用符を区別してください。大量の表データを扱う pandas は、この基礎の後で学びます。

関数：処理に名前を付ける

同じ計算を別の値でも使えるよう、関数にまとめます。

```
def to_minutes(hours):
    return hours * 60

result = to_minutes(2.5)
print(result)
```

結果： 150.0。def は関数の定義、hours は受け取る値の名前（引数）、return は返す値（戻り値）の指定です。定義しただけでは計算は実行されません。to_minutes(2.5) と呼び出すと動きます。

print() は表示、return は呼び出し元へ値を渡す操作です。返された値は次の計算に使えます。

```
print(to_minutes(1) + to_minutes(2))
```

結果： 180。関数内で return の代わりに print だけを使うと、呼び出しの戻り値は None（値がないことを表す値）になります。

演習 6：割合を求める関数

percentage(part, whole) を定義し、part / whole * 100 を返してください。percentage(2, 5) が 40.0 になることを確認します。ここでは whole が正の数であることを前提にします。

エラーを読んで直す

エラーは、処理を進められなかった場所を知らせる情報です。まずメッセージの最後の行で種類を読み、示された行とその前後を確認します。

エラー	典型的な原因	最初に確かめること
NameError	変数が未定義	スペル、定義セルの実行
TypeError	型の組み合わせが不適切	文字列と数値の区別
SyntaxError	文法が不正	:, 括弧、引用符
IndentationError	字下げが不正	ブロックを半角 4 スペースにそろえる
IndexError	リストの範囲外	5 要素なら位置は 0~4

エラー	典型的な原因	最初に確かめること
KeyError	辞書にキーがない	キーの名前
ZeroDivisionError	0 で割った	分母、空のリスト

修正問題： 以下は意図的に誤ったコードです。単独のセルで試し、理由を説明して直します。提出時には修正済みになしてください。

```
print("3" + 2)
```

もう一問： `study_hours = [2, 5, 0, 3, 10]` に対して `study_hours[5]` を使うとどうなるでしょう。最後の値を取り出す方法を二つ考えてください。

💡 AI に相談するとき

まず結果を予想し、自分で 1 回実行します。相談では「期待した結果」「実際の結果」「短いコード」を伝え、「答え全部ではなく、原因とヒントを説明して」と依頼してみましょう。修正後は自分で再実行し、変更理由を説明します。実在の学生情報や認証情報は入力しません。

総合課題：学習時間の集計メモ

次のデータを新しいセルに定義してください。これは小規模な架空データであり、実際の学生集団について推論するための標本ではありません。

```
survey_hours = [2, 5, 0, 3, 10]
```

1. 人数、合計、平均を計算し、単位とともに表示します。
2. 5 時間以上の人数と割合 (%) を求めます。人数は `for` と `if` で数え、割合は演習 6 の関数を使います。
3. Python 標準ライブラリの `statistics` から `median` を読み込み、中央値を求めます。標準ライブラリは Python に付属する道具で、追加インストールは不要です。
4. 平均と中央値の違い、0 時間の扱い、データの限界を Markdown セルに 3~5 文で説明します。
5. ランタイムを再起動し、上からすべて実行して同じ結果になることを確認します。

中央値の計算方法：

```
from statistics import median
print(median(survey_hours))
```

確認値： 5 人、20 時間、平均 4.0 時間、5 時間以上 2 人、40.0%、中央値 3 時間。

提出するものは、コード・結果・考察を含む `.ipynb` ファイルです。Colab のファイルメニューからノートブックをダウンロードできます。AI を利用した場合は、何を相談し、何を自分で確認したかを末尾に記録します。

確認観点	達成の目安
再現性	再起動後も上から実行できる
計算	全確認値が一致し、単位がある
理解	条件と繰り返しの動きを説明できる
解釈	0 と欠測を区別し、過度な一般化をしない

解答例と振り返り

先に自分のコードを保存してから比較してください。同じ結果になる別の書き方も正解です。

演習 1・2

```
book_price = 850
books = 3
shipping = 200
print(book_price * books + shipping)
books = 4
print(book_price * books + shipping)

minutes_text = "90"
hours = int(minutes_text) / 60
print(f"学習時間は{hours}時間です")
```

結果は 2750 円、3600 円、1.5 時間です。冊数を変えた後に合計を再計算しています。

演習 3・4

```
changed_hours = [2, 5, 0, 3, 20]
print(sum(changed_hours) / len(changed_hours))
```

平均は 6.0 時間です。一つの大きな値で平均が上がるため、分布や中央値も確認します。演習 4 で結果が変わるのは 5 時間のときです。>= は 5 を含み、> は含みません。

演習 5・6

```
study_hours = [2, 5, 0, 3, 10]
selected = []
for hours in study_hours:
    if hours >= 5:
        selected.append(hours)
print(selected)

def percentage(part, whole):
    return part / whole * 100

print(percentage(2, 5))
```

結果は [5, 10] と 40.0。修正問題は `print(int("3") + 2)` なら数値の 5 になります。文字の連結を意図するなら `print("3" + "2")` です。最後の要素は `study_hours[4]` または `study_hours[-1]` で取得できます。

総合課題の解答例

```
from statistics import median

def percentage(part, whole):
    return part / whole * 100

survey_hours = [2, 5, 0, 3, 10]
n = len(survey_hours)
total = sum(survey_hours)
count = 0
for hours in survey_hours:
    if hours >= 5:
        count = count + 1

print(f"人数: {n}人")
print(f"合計: {total}時間")
```

```
print(f"平均: {total / n:.1f}時間")
print(f"5 時間以上: {count}人")
print(f"割合: {percentage(count, n):.1f}%")
print(f"中央値: {median(survey_hours)}時間")
```

`:.1f` は小数点以下を 1 桁にそろえて表示する指定です。このコードは空でない数値リストを前提にしています。

考察例：平均は 4.0 時間、中央値は 3 時間である。10 時間という値が平均を高くしている。0 時間は有効な回答として含めた。この架空の 5 人の結果を、大学生全体の傾向と考えることはできない。

振り返り： `=` と `==`、`print` と `return`、リストと辞書の違いを、コードを指しながら相手に説明してください。次は CSV の読み込みと pandas による表データの集計へ進みます。

公式資料

- ・ Python チュートリアル：数値・文字列・リスト
- ・ Python チュートリアル：条件分岐・繰り返し・関数
- ・ Google Colab FAQ
- ・ VS Code での Jupyter ノートブック

教材確認日：2026 年 10 月 2 日。画面の名称はサービスの更新で変わることがあります。